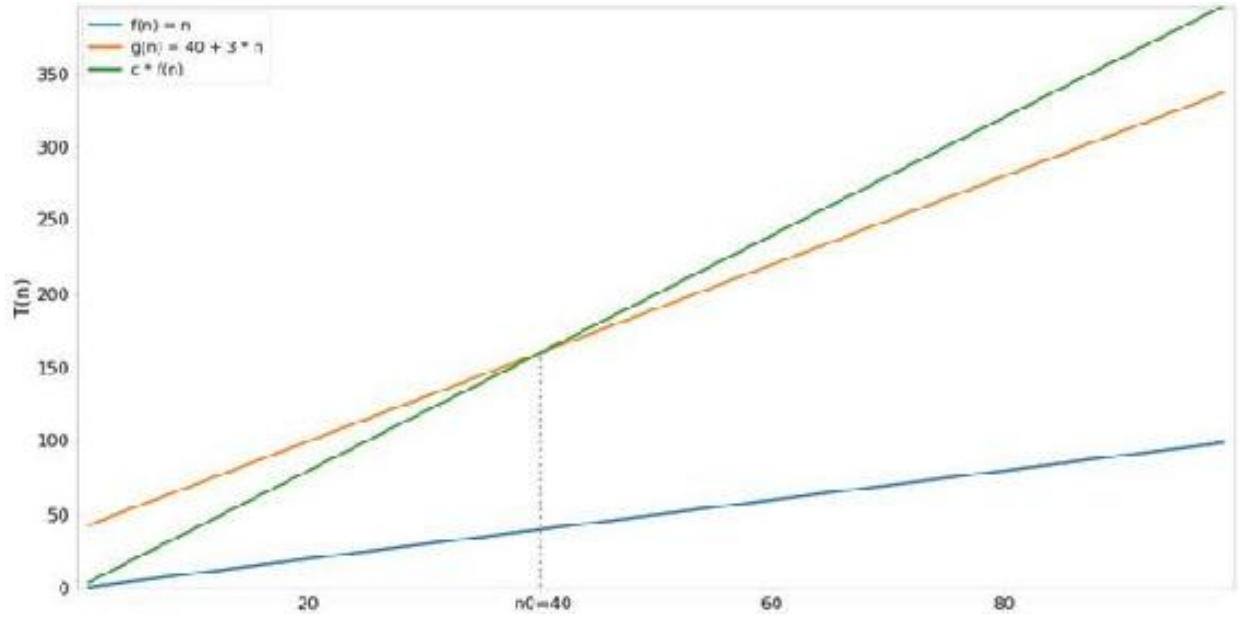


Figure B.2 Visual comparison of functions: `f(n)=n`, `g(n) = 40 + 3*n`, `4*f(n)`. While `f(n)` is always smaller than `g(n)`, `4*f(n)` becomes larger than `g(n)` for sufficiently large values of `n`.

But don't forget that the condition `g(n) ≤ c * f(n)` must hold true for every $n \geq n_0$; we can't rigorously prove that by plotting a chart or plugging in (a finite number of) values for `n` in the formula (see figure B.3 for a counter-example), but we would need to resort to some algebra; nevertheless, plotting the functions can help you get an idea of how the grow and if you are going in the right direction.

Moving back to our example, if we say that algorithm A is `O(n^2)`, it means that `T(n) = O(n^2)`, where `T(n)` is the running time of the algorithm. Or, in other words, algorithm A will never require more than a quadratic number of steps.

The definition above for `O(n)` has some consequences: